Caminhos mínimos

Estruturas de Dados

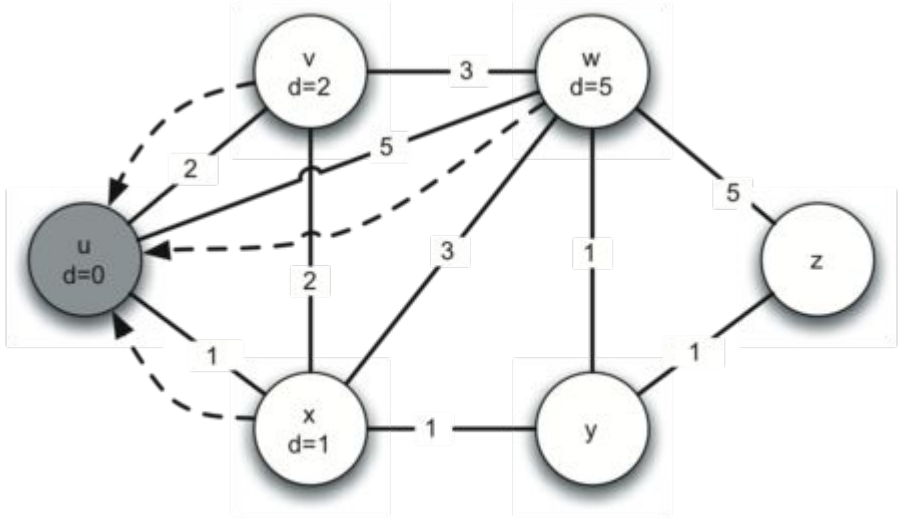


UnB | CIC

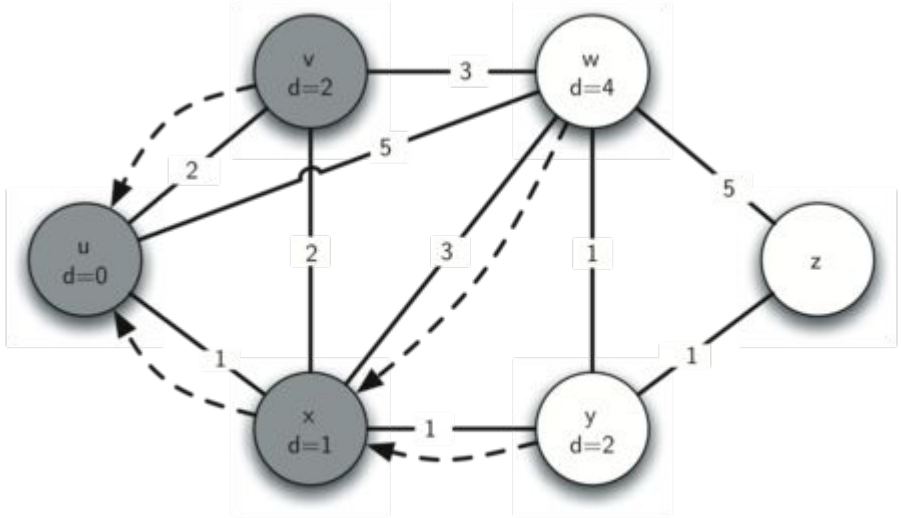
Algoritmo de Dijkstra

- É um algoritmo iterativo que nos fornece o caminho de menor curso de um nó inicial específico para todos os outros nós no grafo.
- É comumente utilizado em grafos dirigidos ou não dirigidos com pesos não negativos nas arestas.
- Seu funcionamento é similar a busca em largura.
- Mantém:
  - O predecessor com menor caminho.
  - O custo total do nó inicial até o nó destino.
  - Faz uso de uma fila de prioridade.

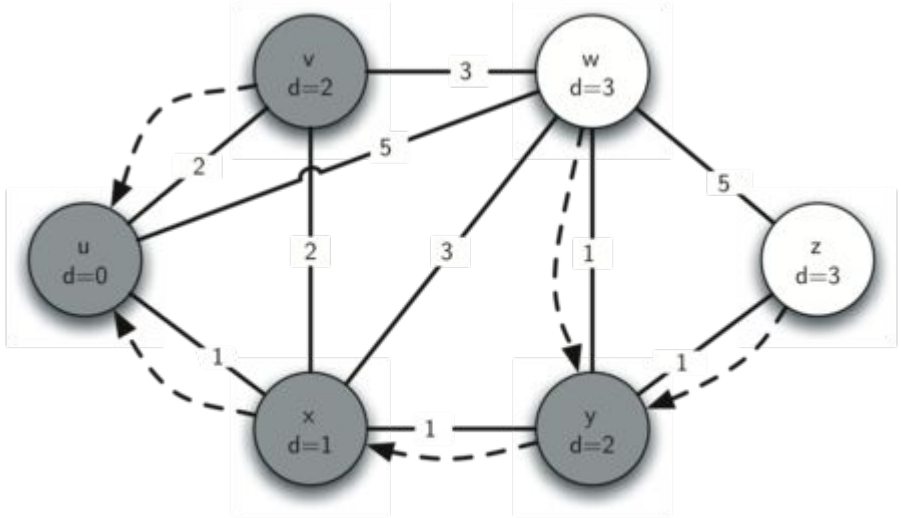
Algoritmo de Dijkstra



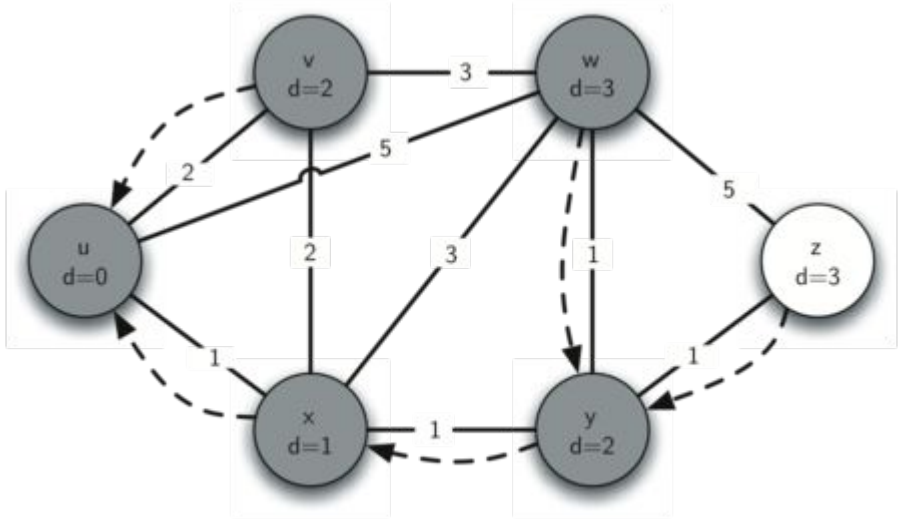
Algoritmo de Dijkstra



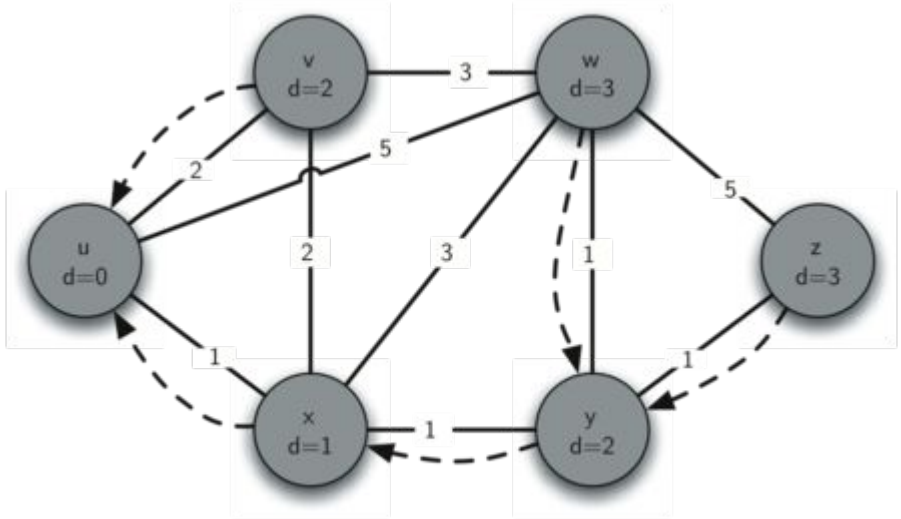
Algoritmo de Dijkstra



Algoritmo de Dijkstra



Algoritmo de Dijkstra



Algoritmo de Dijkstra

```
from pythonds.graphs import PriorityQueue, Graph, Vertex

def dijkstra(aGraph, start):
    pq = PriorityQueue()
    start.setDistance(0)
    pq.buildHeap([(v.getDistance(),v) for v in aGraph])
    while not pq.isEmpty():
        currentVert = pq.delMin()
        for nextVert in currentVert.getConnections():
            newDist = currentVert.getDistance() \
                    + currentVert.getWeight(nextVert)
            if newDist < nextVert.getDistance():
                nextVert.setDistance(newDist)
                nextVert.setPred(currentVert)
                pq.decreaseKey(nextVert,newDist)
```

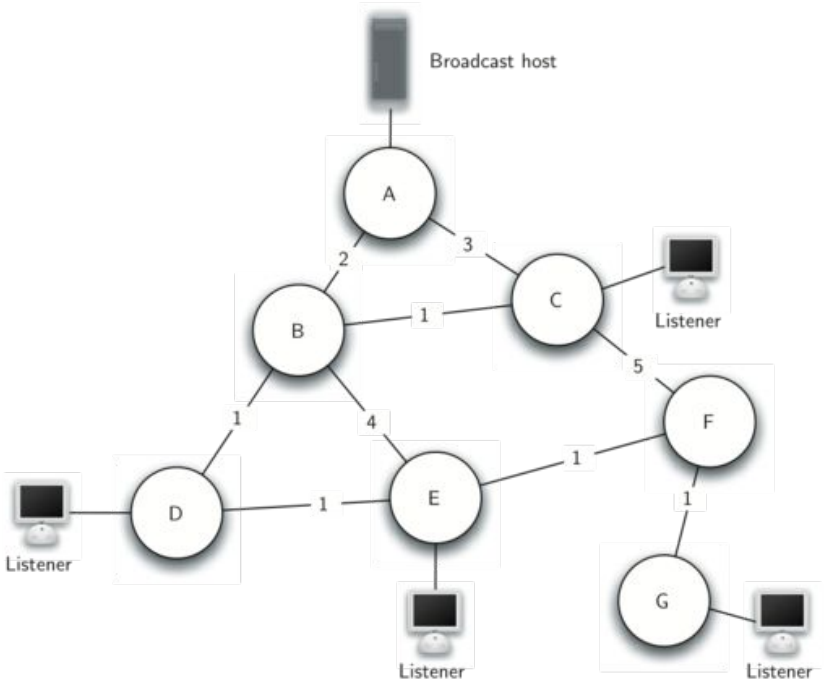
Algoritmo de Dijkstra

```
from pythonds.graphs import PriorityQueue, Graph, Vertex

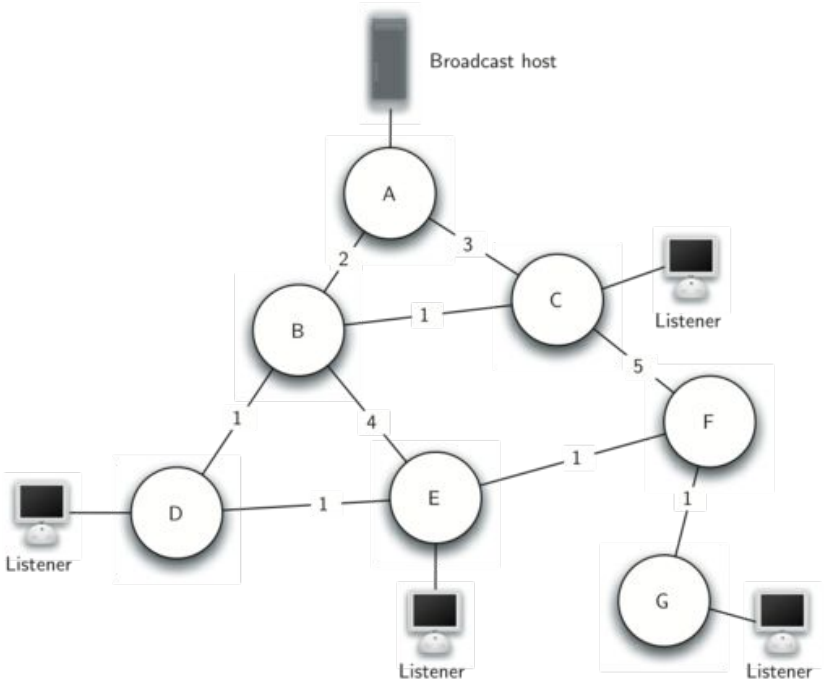
def dijkstra(aGraph, start):
    pq = PriorityQueue()
    start.setDistance(0)
    pq.buildHeap([(v.getDistance(),v) for v in aGraph])
    while not pq.isEmpty():
        currentVert = pq.delMin()
        for nextVert in currentVert.getConnections():
            newDist = currentVert.getDistance() \
                    + currentVert.getWeight(nextVert)
            if newDist < nextVert.getDistance():
                nextVert.setDistance(newDist)
                nextVert.setPred(currentVert)
                pq.decreaseKey(nextVert,newDist)
```

O(V)
O(log V)
O(V log V)
O(E log V)
O((V+E) log V)

Prim's Spanning Tree

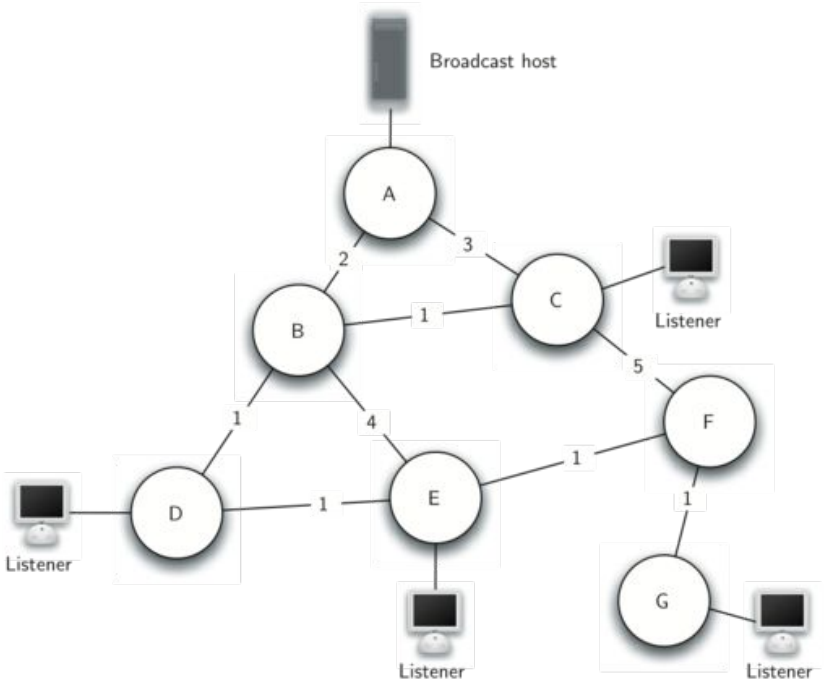


Prim's Spanning Tree



Possíveis soluções:

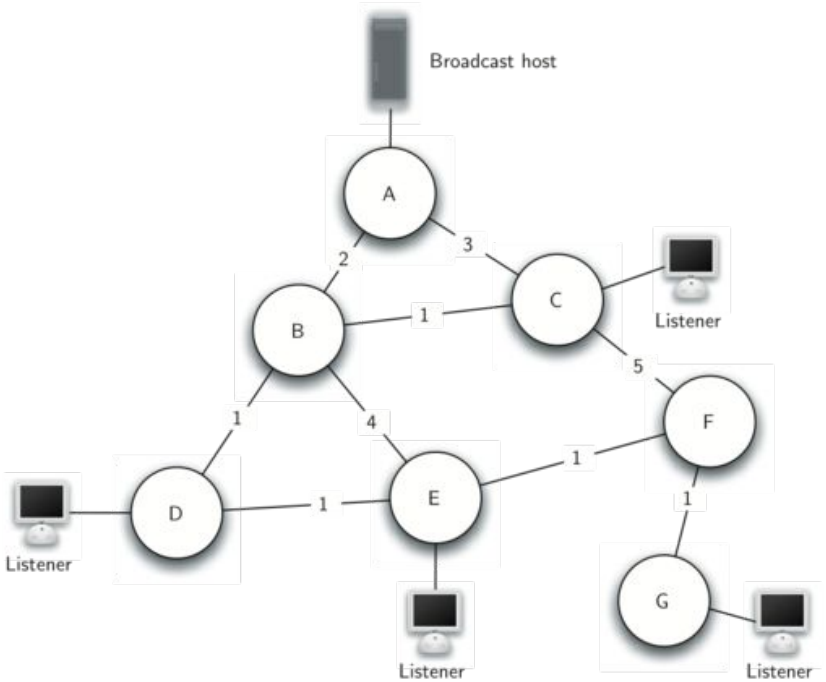
Prim's Spanning Tree



Possíveis soluções:

1. O host de transmissão manter uma **lista de todos os ouvintes** e enviar **mensagens individuais** para cada um. Usando esta abordagem, seriam enviadas quatro cópias de cada mensagem. Supondo que o caminho de **menor custo** seja usado, a mensagem seria **replicada diversas vezes**.

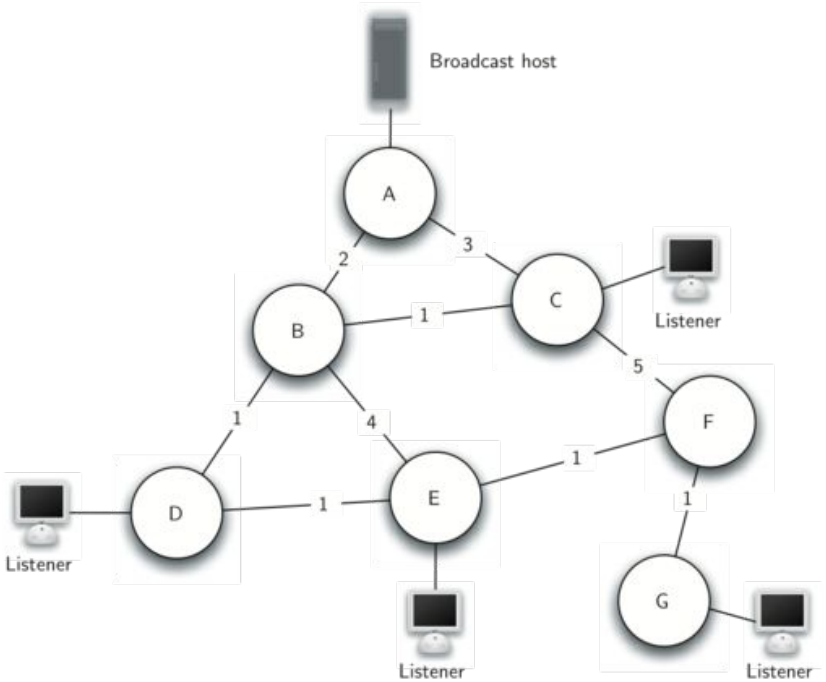
Prim's Spanning Tree



Possíveis soluções:

2. O host de transmissão enviar uma única cópia da mensagem de transmissão com um **tempo de viver** associado e deixar que os roteadores resolvam. Pode gerar uma **inundação descontrolada**.

Prim's Spanning Tree



Possíveis soluções:

3. Minimum weight Spanning Tree (MST)

Minimum weight Spanning Tree

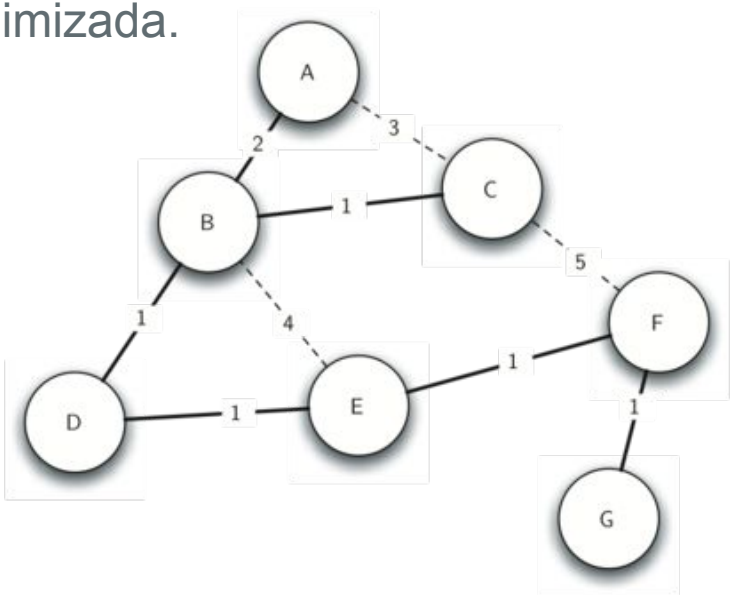
Dado um grafo **G = (V, E)**

- **T** é um subconjunto acíclico de **E** que conecta todos os vértices em **V**. A soma dos pesos das arestas em **T** é minimizada.

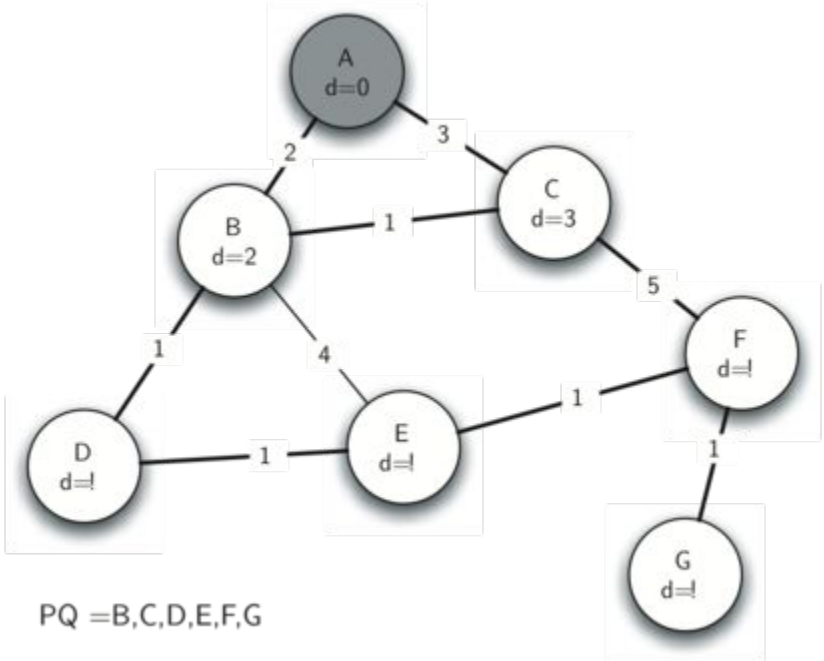
Minimum weight Spanning Tree

Dado um grafo **G = (V, E)**

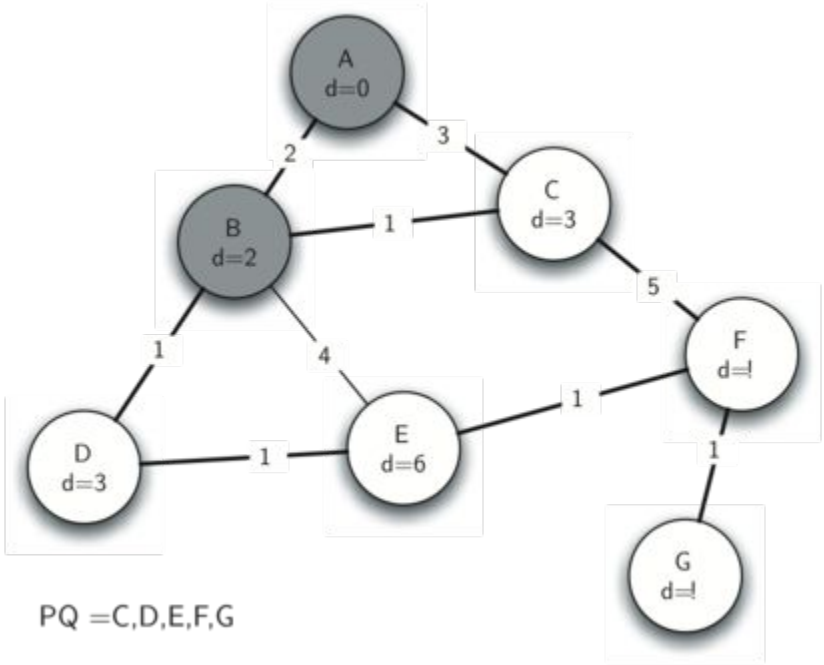
- **T** é um subconjunto acíclico de **E** que conecta todos os vértices em **V**. A soma dos pesos das arestas em **T** é minimizada.



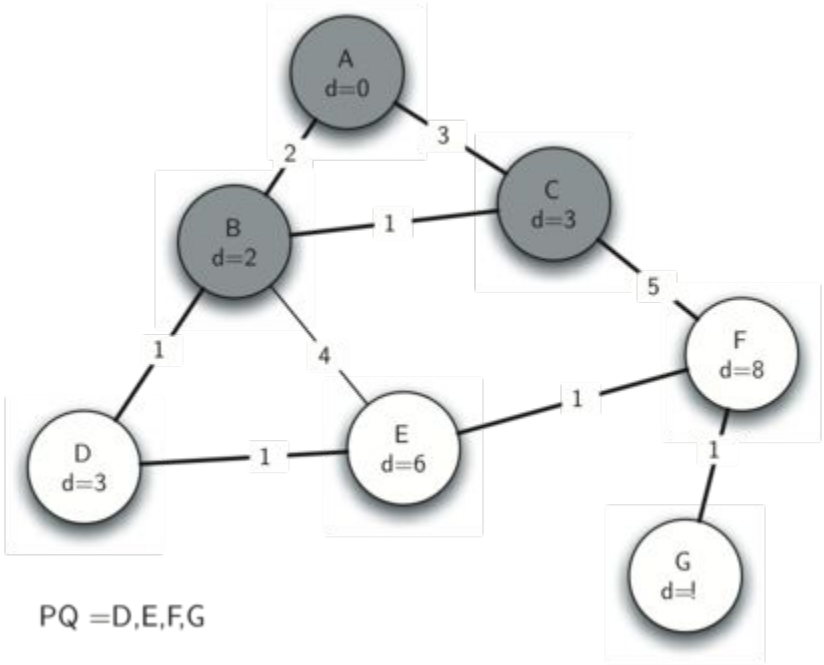
Prim's Spanning Tree



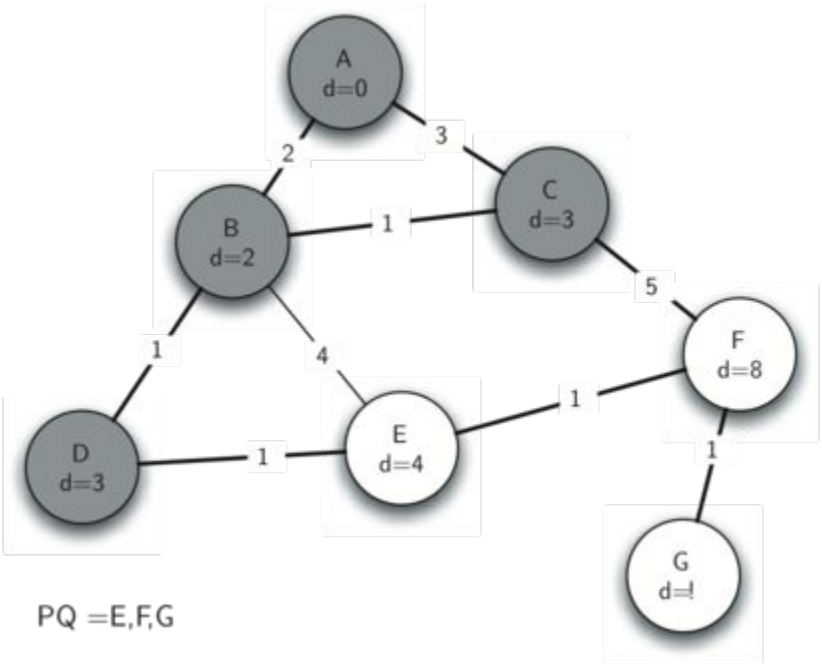
Prim's Spanning Tree



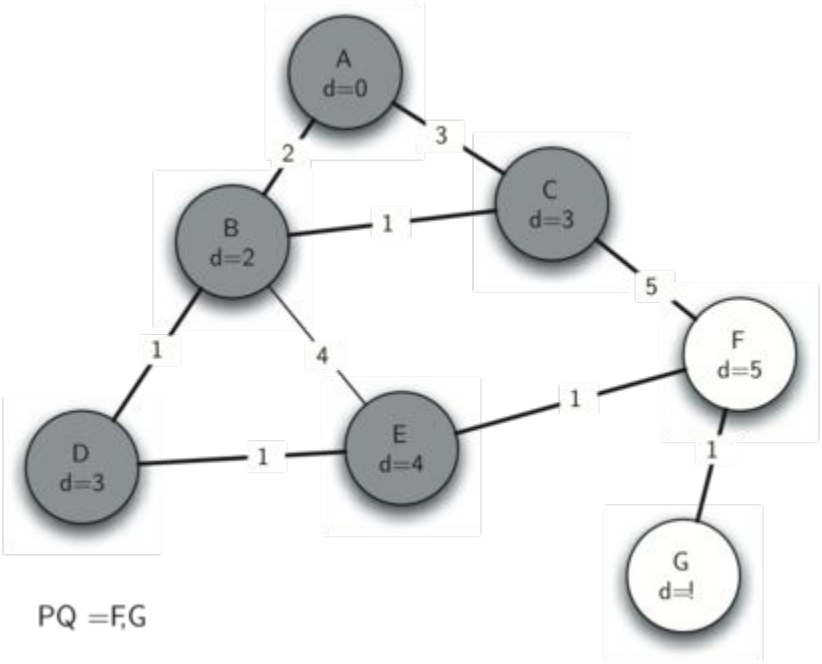
Prim's Spanning Tree



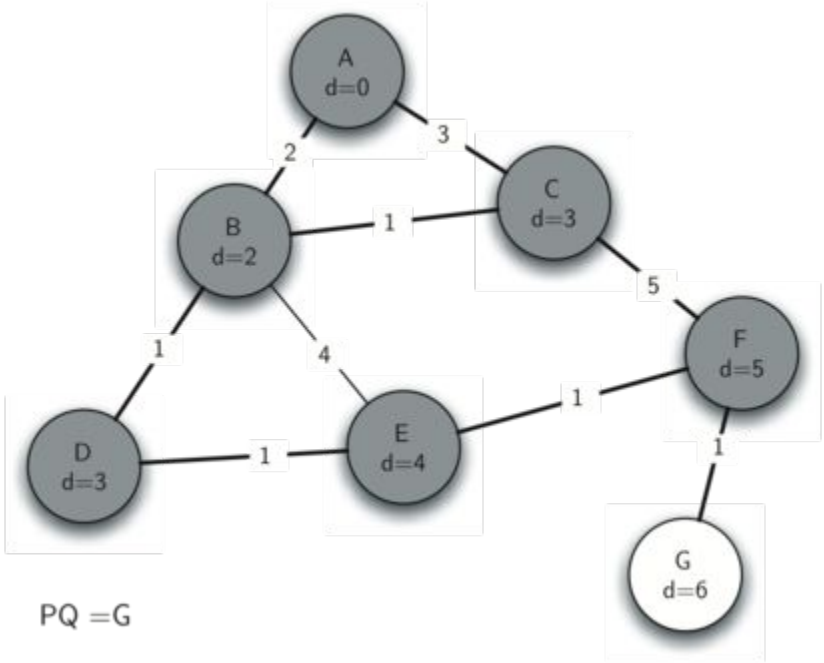
Prim's Spanning Tree



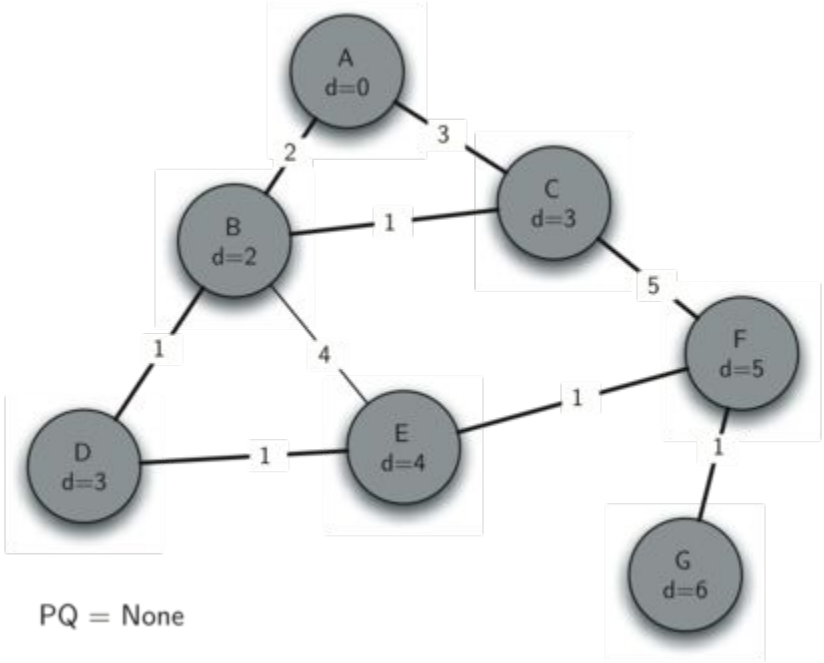
Prim's Spanning Tree



Prim's Spanning Tree



Prim's Spanning Tree



Algoritmo de Prim

```
from pythonds.graphs import PriorityQueue, Graph, Vertex

def prim(aGraph, start):
    pq = PriorityQueue()
    start.setDistance(0)
    pq.buildHeap([(v.getDistance(),v) for v in aGraph])
    while not pq.isEmpty():
        currentVert = pq.delMin()
        for nextVert in currentVert.getConnections():
          newCost = currentVert.getWeight(nextVert)
          if nextVert in pq and newCost<nextVert.getDistance():
              nextVert.setPred(currentVert)
              nextVert.setDistance(newCost)
              pq.decreaseKey(nextVert,newCost)
```